



NTU Academy for Professional
and Continuing Education

(SCTP) Advanced Professional Certificate **Data Science and AI**





NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

3.7 Neural Network and Deep Learning

Module Overview

3.1 Probability and Statistics for Machine Learning

3.2 Introduction to Machine Learning

3.3 Supervised Learning

3.4 Supervised Learning - Advanced

3.5 Unsupervised Learning

3.6 Time Series Data & Forecasting

3.7 Neural Network & Deep Learning

3.8 Computer Vision (CV)

3.9 Natural Language Processing (NLP)

3.10 NLP - Advanced

Lesson Objectives

- The definition, historical overview and applications of Neural Network and Deep Learning
- Perceptron and Multi-Layer Perceptron (MLP) as the foundation of Neural Network
- PyTorch as a Deep Learning framework

Lesson Plan

- Gradient Descent
- Introduction to Neural Networks and Deep Learning
- Perceptron and MLP
- Components of neural network
- Loss functions
- Forward + backward
- Notebook: Building a Fully Connected Neural Network on MNIST and Titanic datasets.

$$MSE = L(y, \underbrace{\beta_0, \beta_1, \dots, \beta_n}) \quad y \sim \beta_0, \beta_1, \beta_2 \dots \beta_n$$

Gradient Descent

$$x = 100$$
$$x \sim 0$$

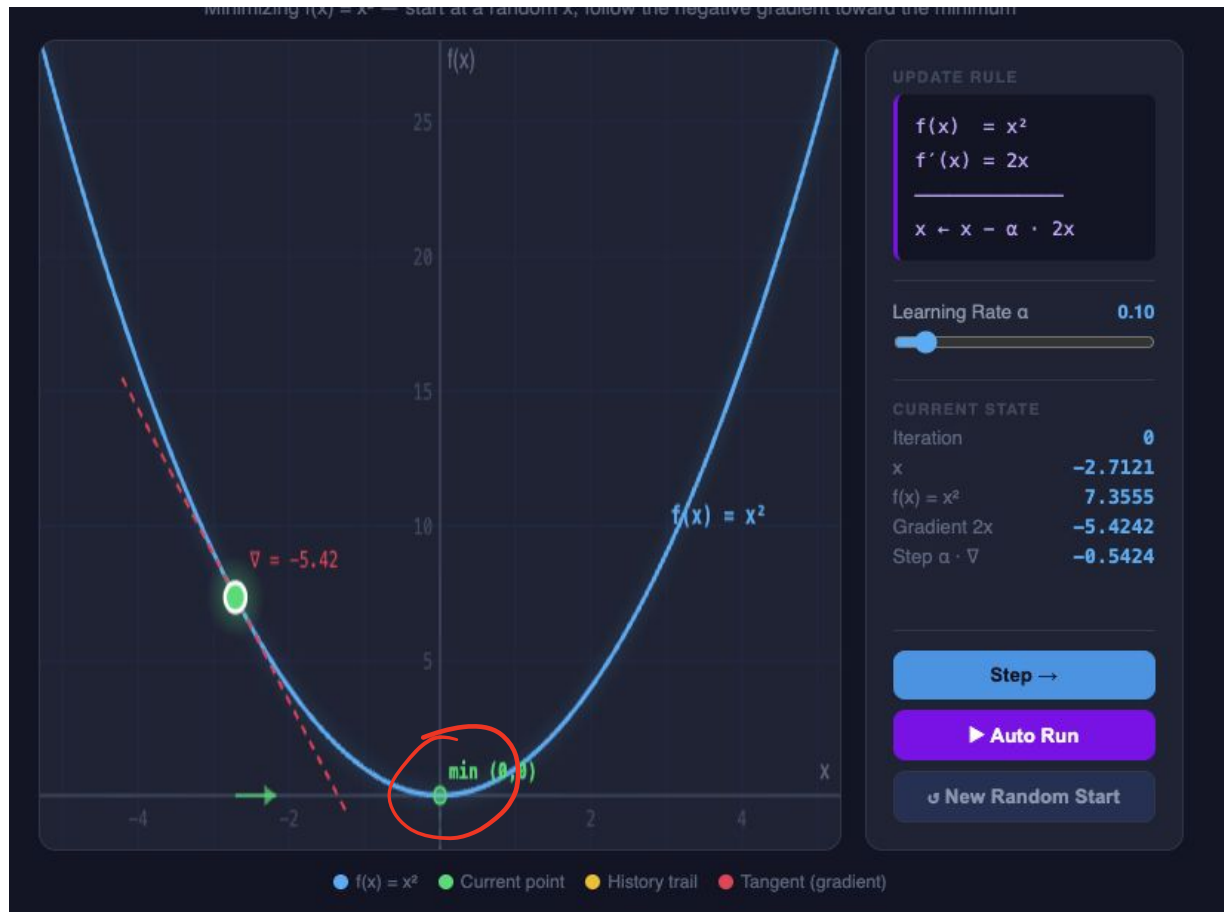
$$(2) \quad x_t = x_{t-1} - \alpha \frac{df}{dx}$$

find x s.t.
minimize
 f

(1) differentiable

$$f = x^2$$
$$\frac{df}{dx} = 2x$$

(3)



minimize
loss

Gradient Descent

- E.g in OLS, instead of requiring a closed analytical solution, we can optimize some Loss w.r.t parameters I.e find the locally best optimal parameters that minimize some loss

$$L = (y - \hat{y})^2$$
$$L = (y - f(\hat{w}))^2$$

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\beta}$$

$\hat{\beta}$ = ordinary least squares estimator

$\mathbf{X}$ = matrix regressor variable $\mathbf{X}$

$\top$ = matrix transpose

$\mathbf{y}$ = vector of the value of the response variable

$\hat{\mathbf{y}}$ = predicted values

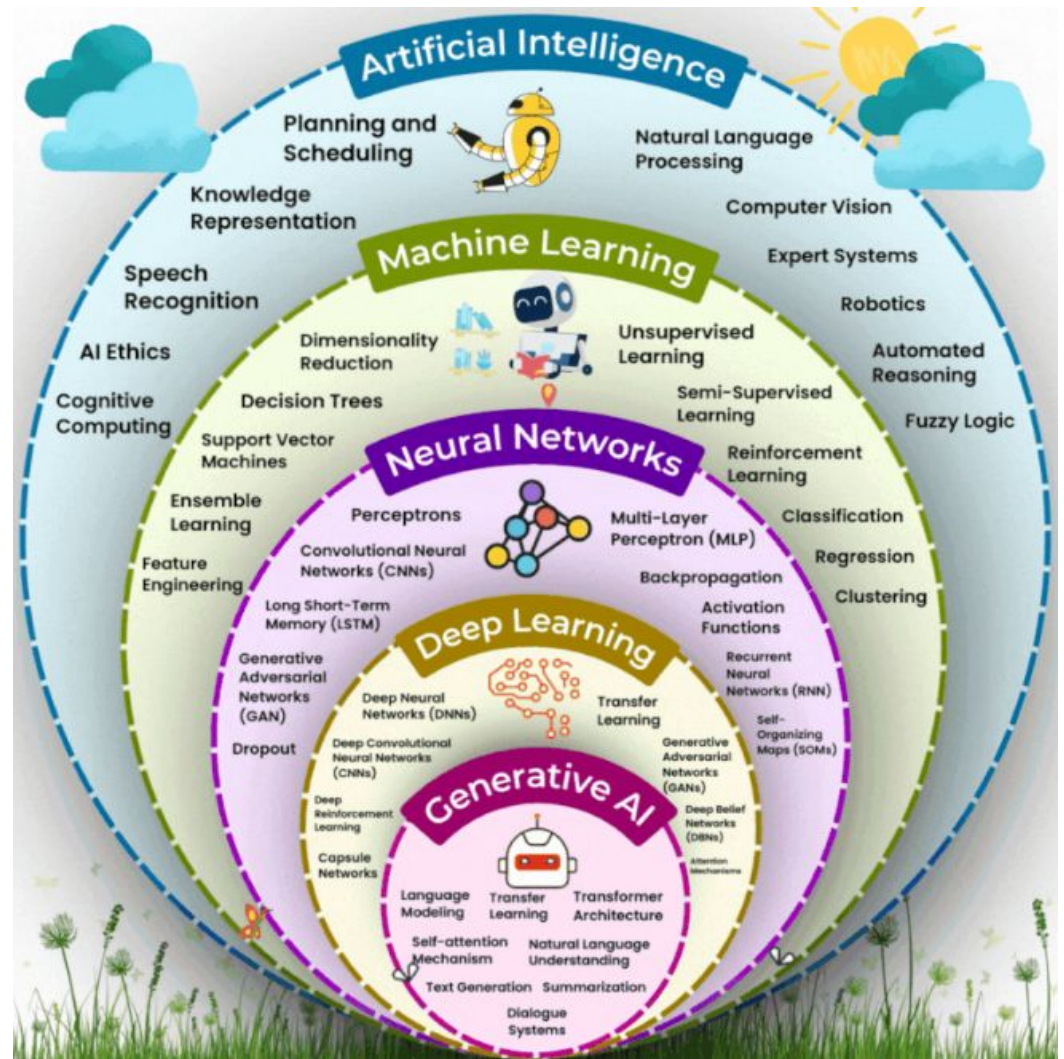


Neural Network vs Deep Learning

$$\textcircled{y} \quad \underbrace{f(x)}_{f(\cdot)} \rightarrow \hat{y}$$

$$y = mx + b$$

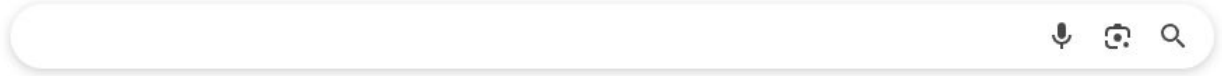
$$\textcircled{f(x, w)} \rightarrow w \cdot x + b$$



Deep Learning in Action



Google



All Images Books Videos News Shopping Maps : More

Tools

THE PASTRY A.I. THAT
LEARNED TO FIGHT
CANCER [link](#)



Rides



Freight

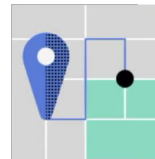


Eats

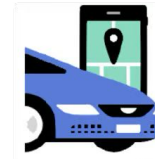


ETAs power **Uber** experiences [link](#)

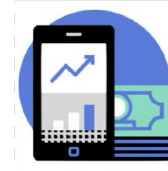
Dispatch



Navigation



Fares & many more



Neural Network vs Deep Learning

Neural Network

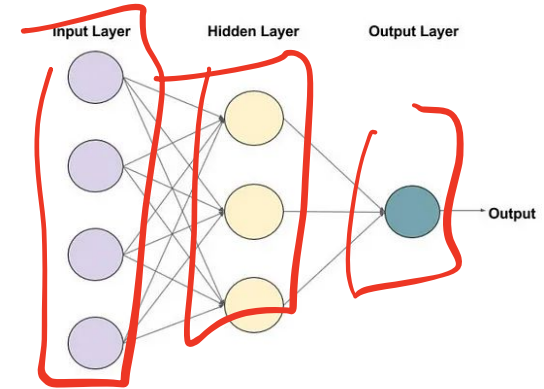
A neural network is a computational model inspired by the structure of the human brain. It consists of layers of nodes ("neurons") where each neuron receives input, processes it (typically using a weighted sum and activation function), and passes it on to the next layer.

Basic structure:

- Input layer
- Hidden layer(s)
- Output layer

Use cases:

Basic pattern recognition, regression, classification, etc



Neural Network vs Deep Learning

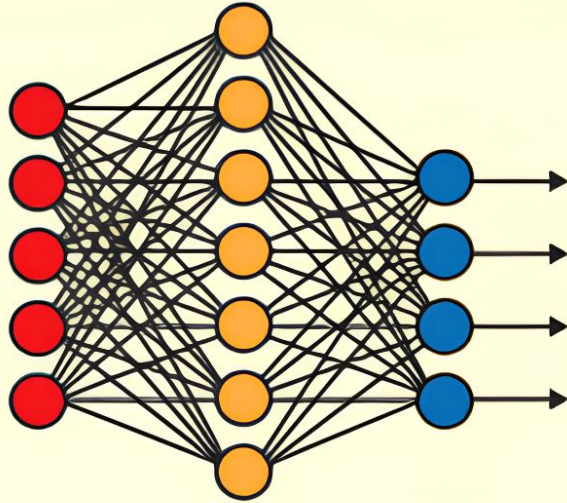
Deep Learning

Deep learning is a subset of neural networks that uses neural networks with many layers (hence "deep"). The term "deep" refers to the depth of the network, i.e., the number of hidden layers.

- ~~Deep Neural Networks (DNNs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs)~~ are all types of deep learning models.
- It excels at automatically learning features from large amounts of unstructured data like images, audio, and text. (LLM)
- Teaching a computer how to learn a task just from raw data

inductive Big 5

Simple Neural Network



3 B

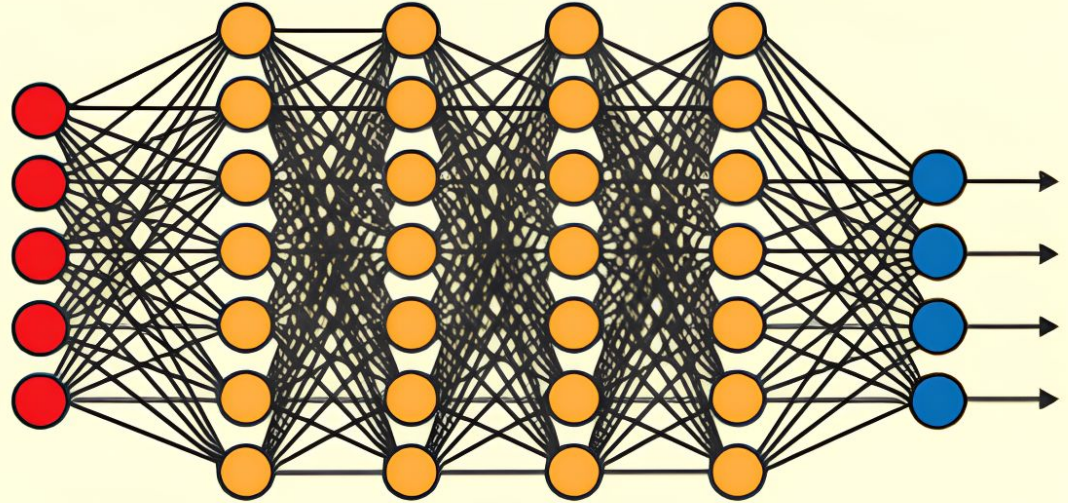
● Input Layer

● Hidden Layer

● Output Layer



Deep Learning Neural Network

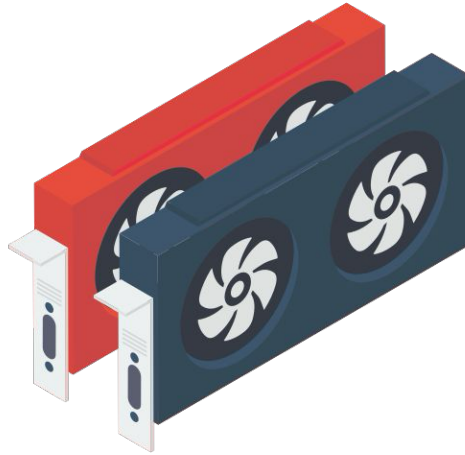


Why now?

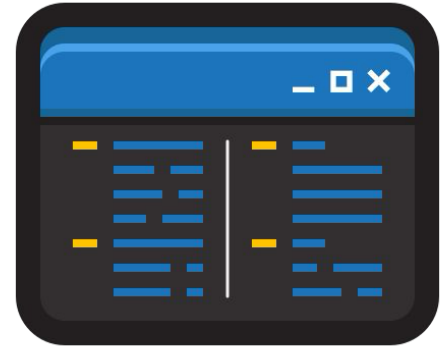
Neural networks dates way back, so why did it only get more popular in the recent years?



Big Data



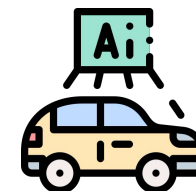
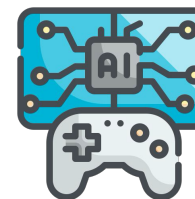
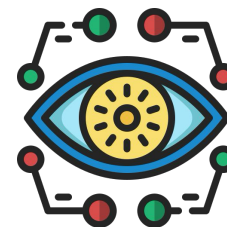
Hardware

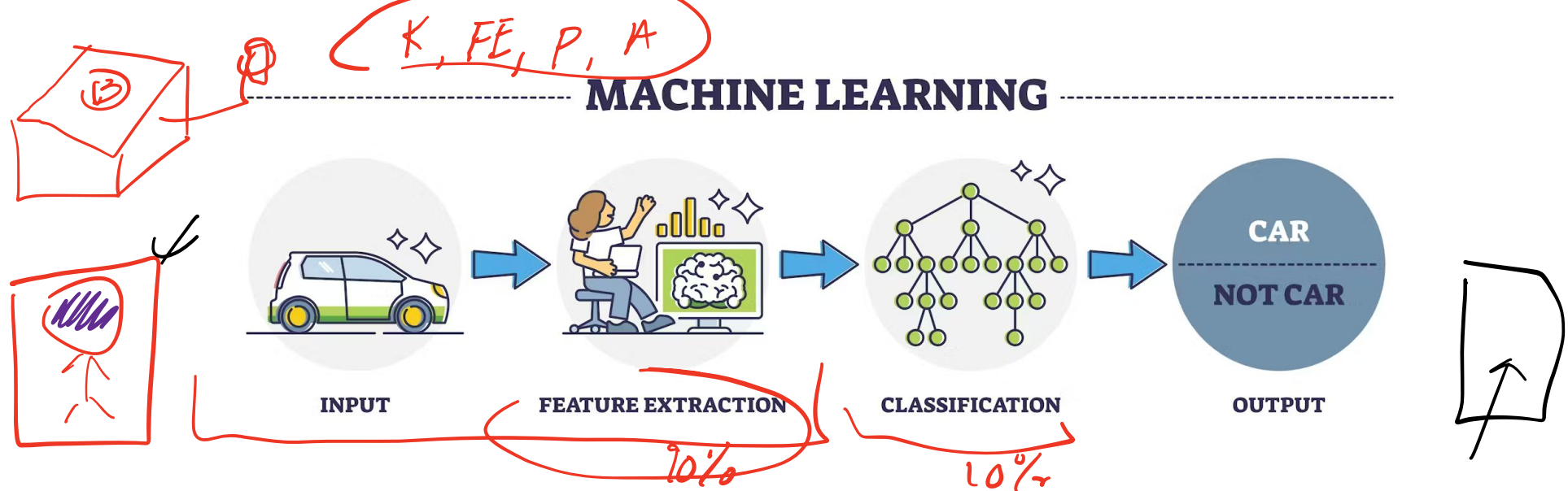


Software

Applications of Deep Learning

- **Computer Vision:** Image and video recognition, object detection, image generation, and facial recognition
- **Natural Language Processing (NLP):** Language translation, sentiment analysis, and chatbots (including LLMs and ChatGPT)
- **Speech Recognition:** Voice control systems, transcription, and real-time translation
- **Healthcare:** Disease detection, drug discovery, and personalized medicine
- **Autonomous Vehicles:** Self-driving cars and drones
- **Gaming:** Non-player character (NPC) behavior, procedural content generation, and game testing
- **Finance:** Fraud detection, algorithmic trading, and credit scoring
- **Robotics:** Humanoid robots, industrial automation, and service robots



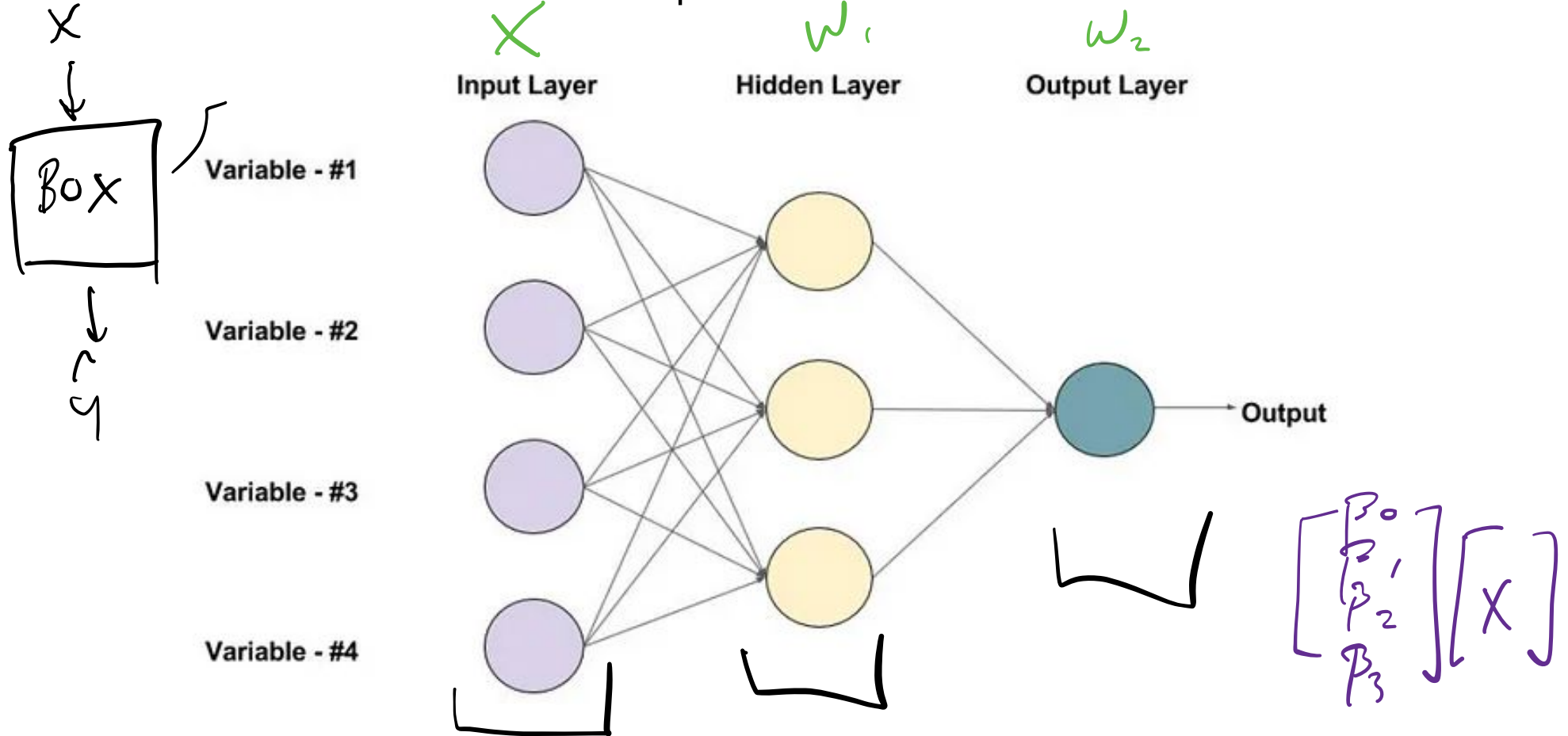


Remember Feature Engineering?

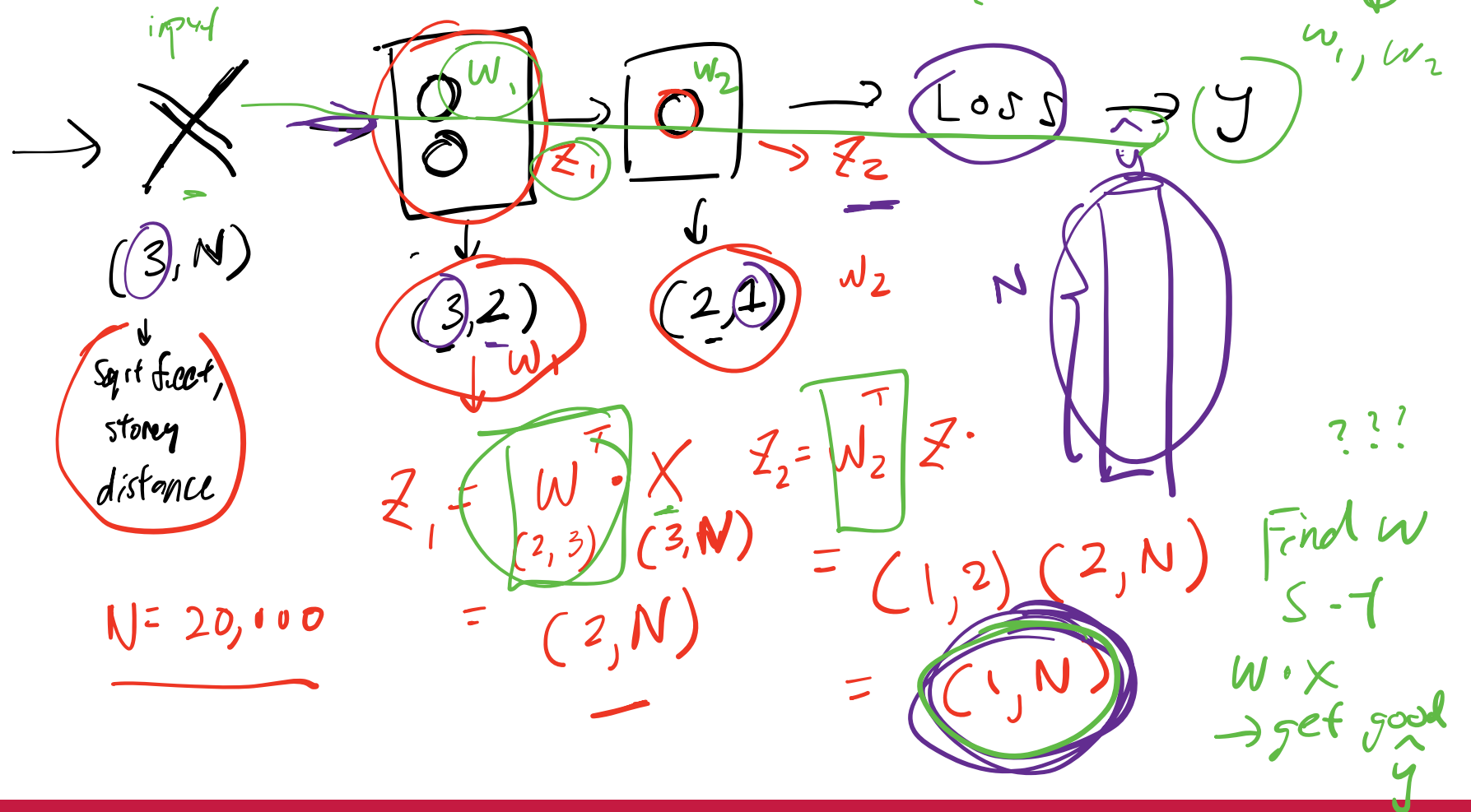
- Feature engineering is the process of selecting, transforming, or creating the most relevant variables, known as “features”, from raw data to use in machine learning models.
- For example, if you're building a weather prediction model, the raw data might include temperature, humidity, wind speed, and barometric pressure.
- Feature engineering would involve determining which of these variables are most important for predicting the weather and possibly transforming them (e.g., converting temperature from Fahrenheit to Celsius) to make them more useful for the model.
- In traditional machine learning, feature engineering is often a manual and time-consuming process that requires domain expertise. However, one of the advantages of deep learning is that it can automatically learn relevant features from the raw data, reducing the need for manual intervention.



Neural Network Components



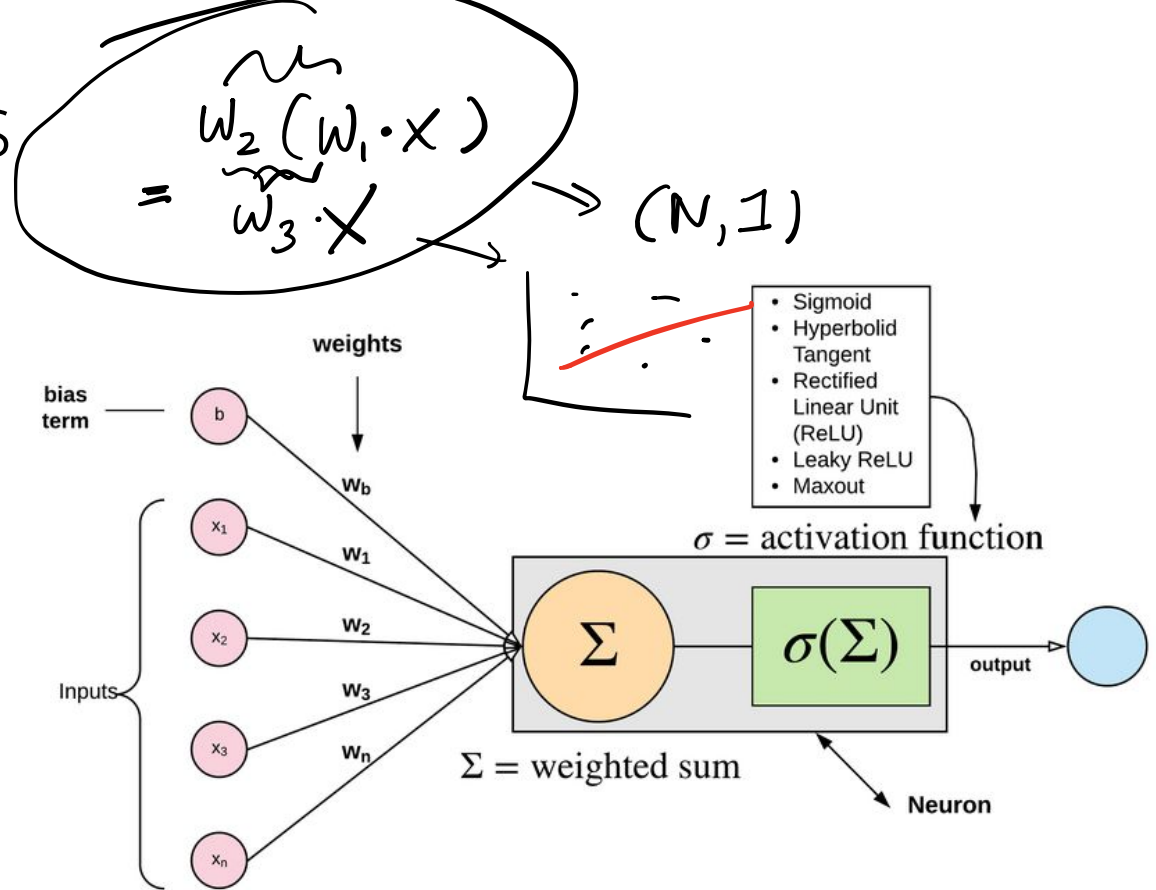
Feed Forward Neural Network



Activation Functions

Non-linear activation functions are crucial in MLPs and neural networks, as they allow the network to capture complex patterns and interactions in the data. Without non-linearity, a neural network, regardless of how many layers it had, would still behave like a single-layer perceptron, only capable of modeling linear separations.

Purpose of an activation function is to add non-linearity to the neural network



W

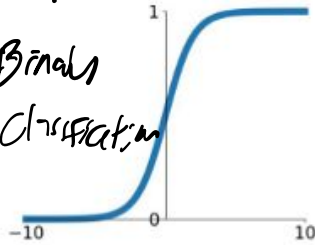
Activation Functions

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

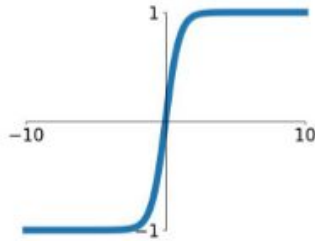
last layer

Binary
classification



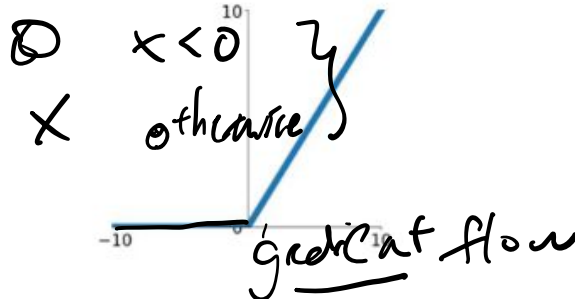
tanh

$$\tanh(x)$$



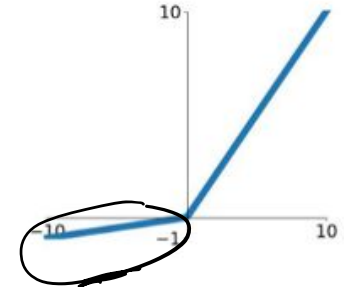
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

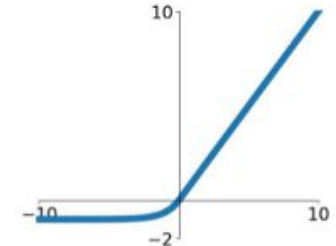


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

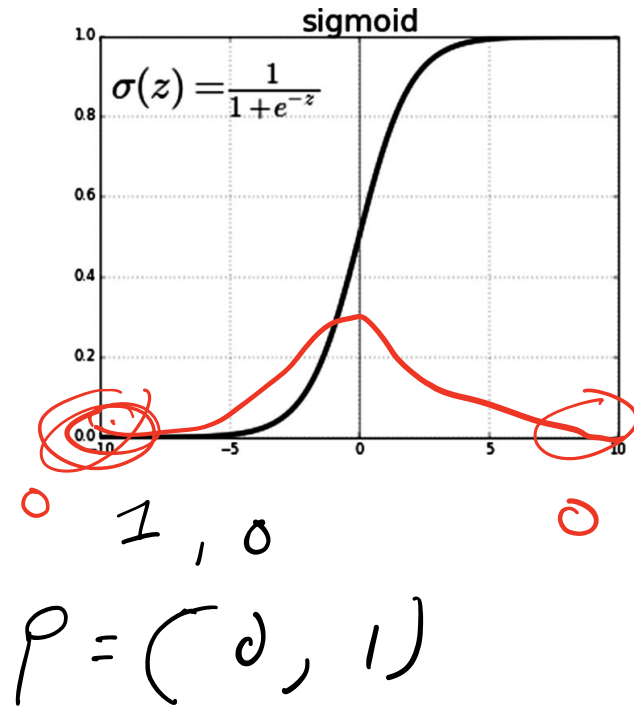
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Sigmoid Activation Function

The sigmoid/logistic activation function is one of the most widely used functions:

- It is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice because of its range.
- The function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. This is represented by an S-shape of the sigmoid activation function. A smooth gradient means the function has a continuous and non-jumping slope across its domain. This is important for stable learning. If a function had abrupt changes (jumps) in its output, the gradient would be unpredictable, which would make training the model difficult and unstable.
- The sigmoid function has a smooth and continuous gradient, meaning small changes in the input will cause small changes in the output, which helps in making gradual, consistent updates to the model weights during training.

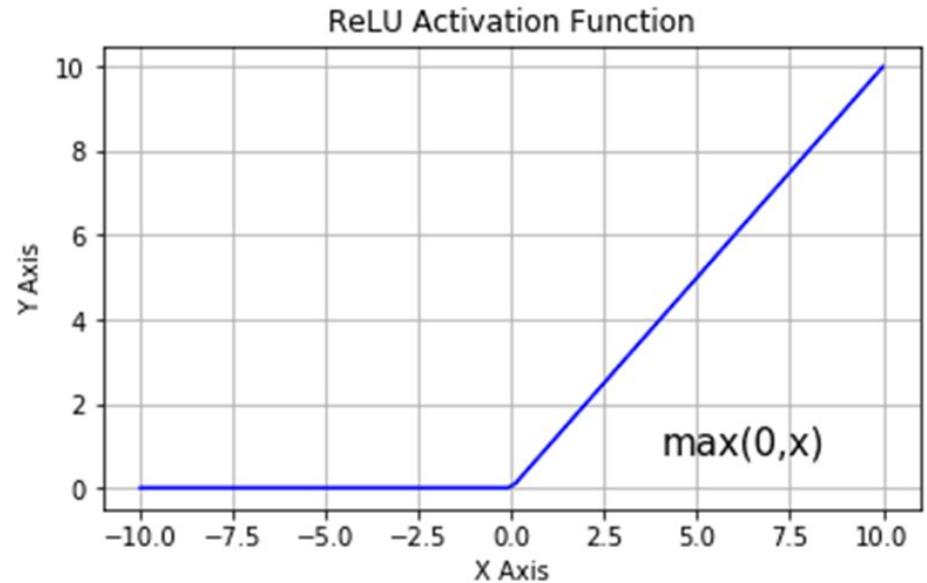


https://link.springer.com/chapter/10.1007/978-981-16-2248-9_5

ReLU Activation Function

ReLU stands for Rectified Linear Unit.

- Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient.
- The main catch here is that the ReLU function does not activate all the neurons at the same time.
- The neurons will only be deactivated if the output of the linear transformation is less than 0.



<https://www.nomidl.com/deep-learning/what-is-relu-and-sigmoid-activation-function/>

Activation Functions at work

important CV

As a rule of thumb, begin with using the ReLU activation function and then move over to other activation functions if ReLU doesn't provide optimum results.

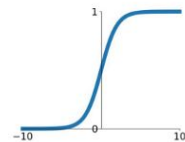
And here are a few other guidelines:

ReLU activation function should only be used in the hidden layers.

- **Sigmoid/Logistic** and **Tanh** functions should not be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).
- **Swish** function is used in neural networks having a depth greater than 40 layers.

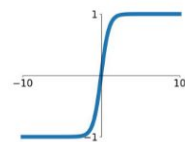
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



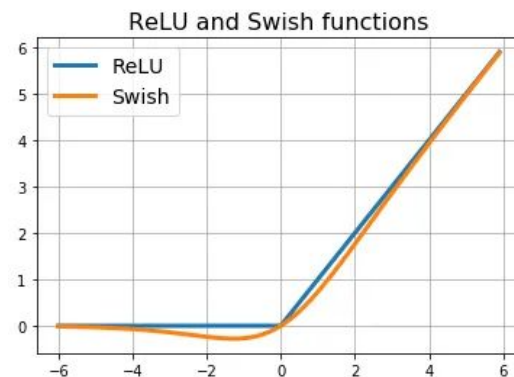
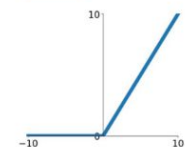
tanh

$$\tanh(x)$$



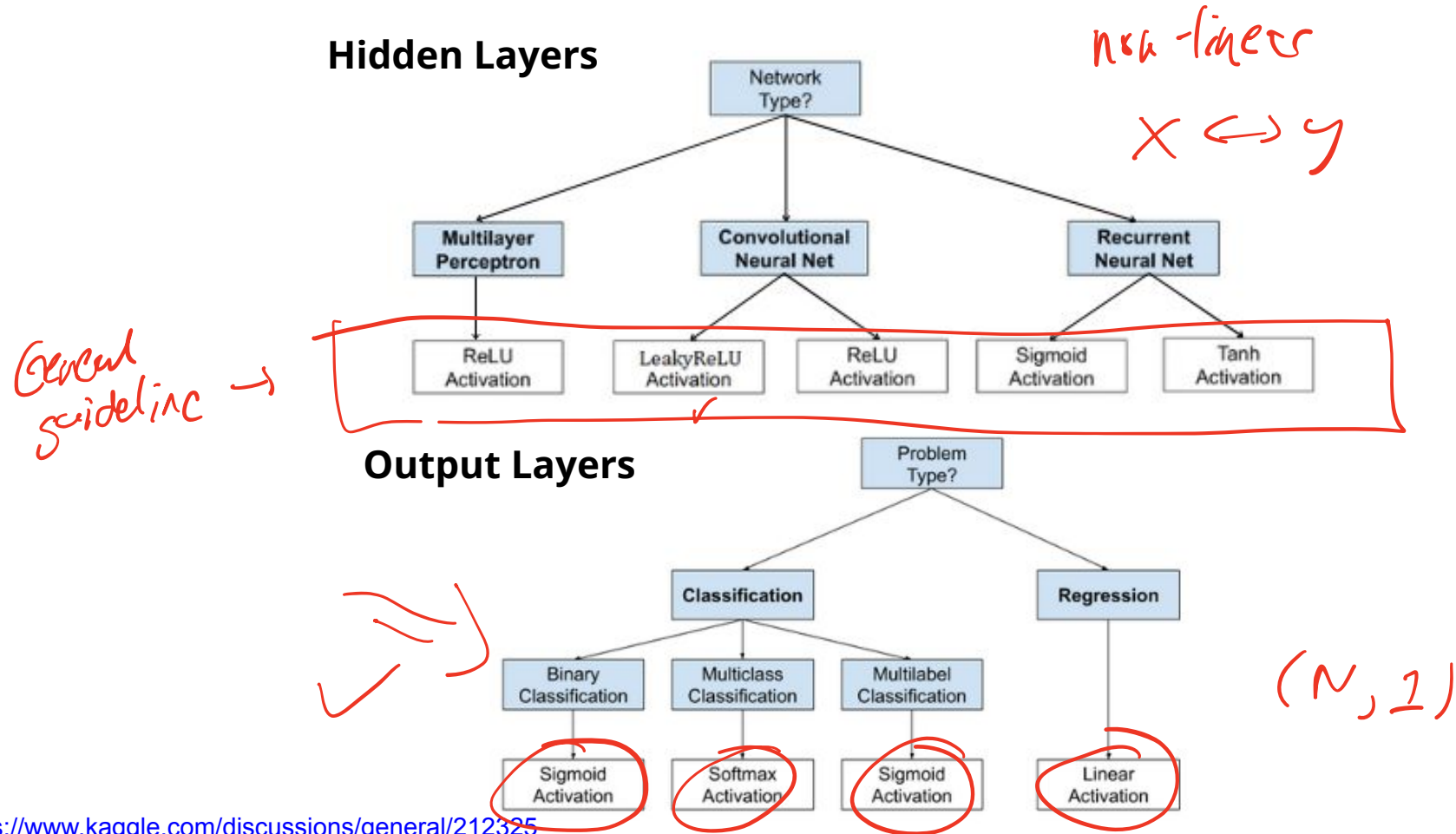
ReLU

$$\max(0, x)$$



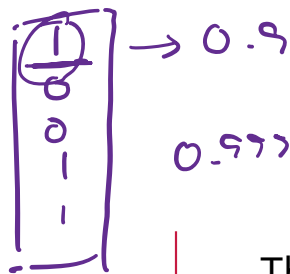
Activation Functions at work

Hidden Layers



Loss Function

The loss function is the difference between actual and predicted values. It allows neural networks to track the model's overall performance. Depending on specific problems, we chose a certain type of function, for example, mean squared error.



Find w??

Best w? How to measure good/Bad

The most used loss functions in deep learning are:

Regression

- Mean squared error
- Mean absolute error
- Huber

Classification

- Binary cross-entropy
- Categorical hinge
- Sparse categorical cross-entropy

$$Loss = \sum (Predicted - Actual)^2$$

gradient

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C t_{ij} \log(P_{ij})$$

going over N

↓ P

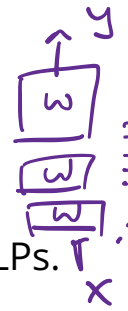
$$-\log(0.9)$$

$$-(1-j \log 0.1)$$

$$-\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C t_{ij} \log(P_{ij})$$

Backpropagation Training

$$(w \cdot w \cdot x)$$



Backpropagation is the algorithm used for training neural networks, including MLPs.

Forward Pass (Feed-Forward)

The input data is passed through the network layer by layer until the output layer is reached.

The output is then used to compute the loss function, which measures the difference between the network's prediction and the actual target values.

A common choice for the loss function in classification tasks is the cross-entropy loss.

Backward Pass (Backpropagation)

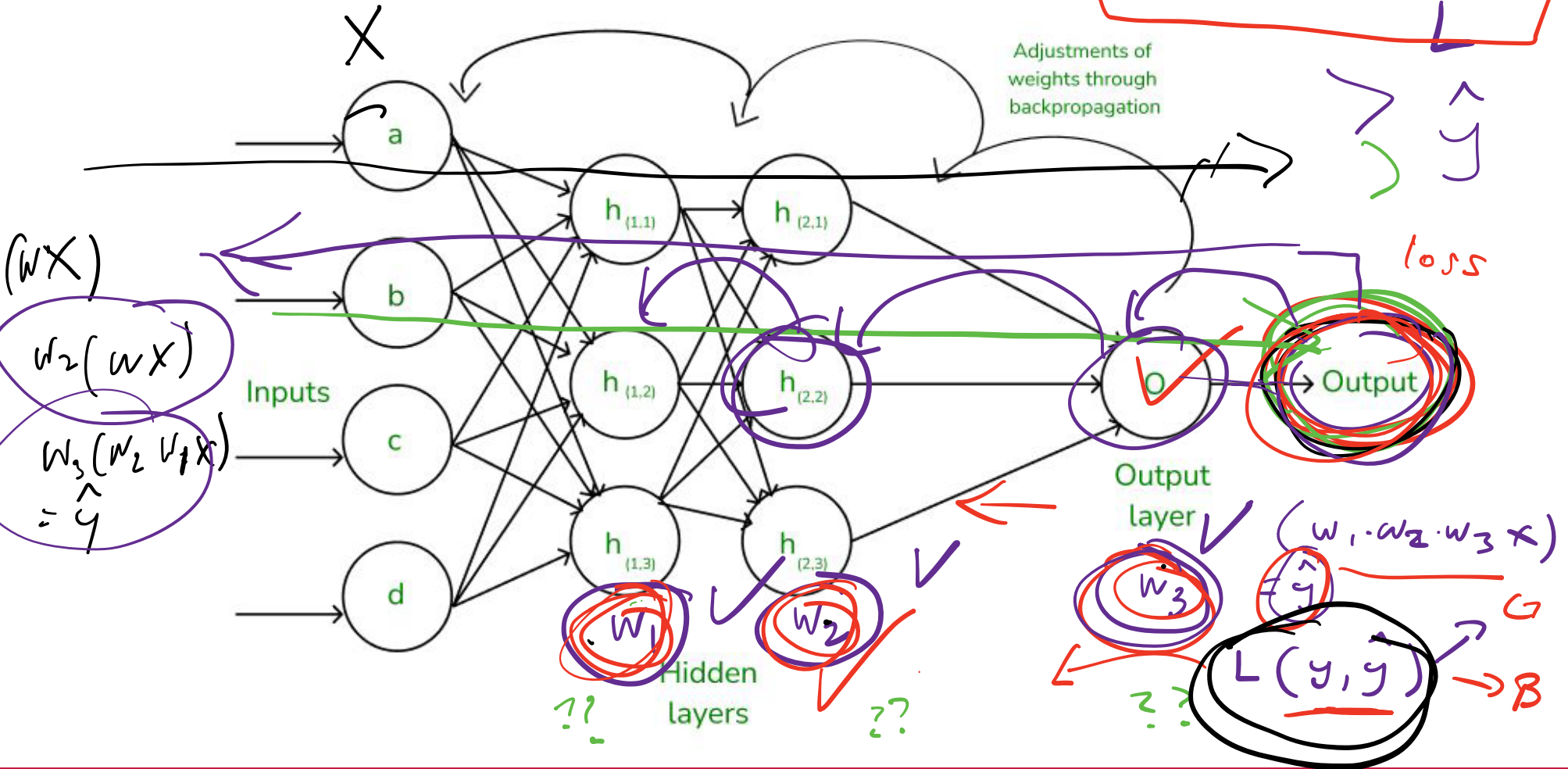
Gradients of the loss function with respect to the network's weights are computed using the chain rule of calculus, in a process known as gradient descent.

Backpropagation allows neural networks to learn from their mistakes, adjusting the weights to improve performance on the training data.

With the right adjustments over many iterations, the network can converge to a state where it makes accurate predictions on unseen data.

Backpropagation Training $\begin{bmatrix} y \\ \vdots \end{bmatrix}$

Find $\underline{W}$ s.t. $L(W, x)$
minimize $\underline{L}$



Backpropagation Training

$$GD \rightarrow x_t = x_{t-1} - \frac{df}{dx_{t-1}}$$

Find x s-f

$f(x)$ is minimize

learning rate

0.01

w_i

$$w_t = w_{t-1} - \frac{dL}{dw}$$

Find w s-f

L is minimize

w gets better

How to get $\frac{dL}{dw}$?

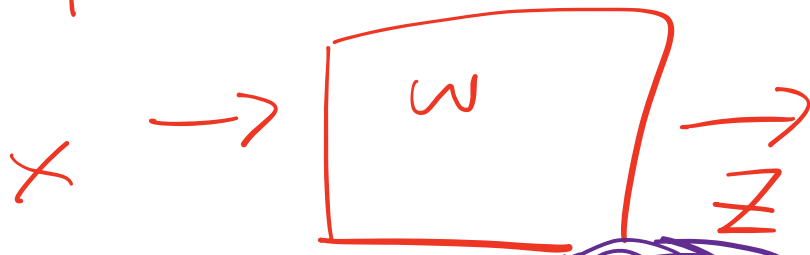
$$Z = w \cdot x \quad \frac{dZ}{dw} = \frac{dZ}{dx} \cdot x$$

chain rule

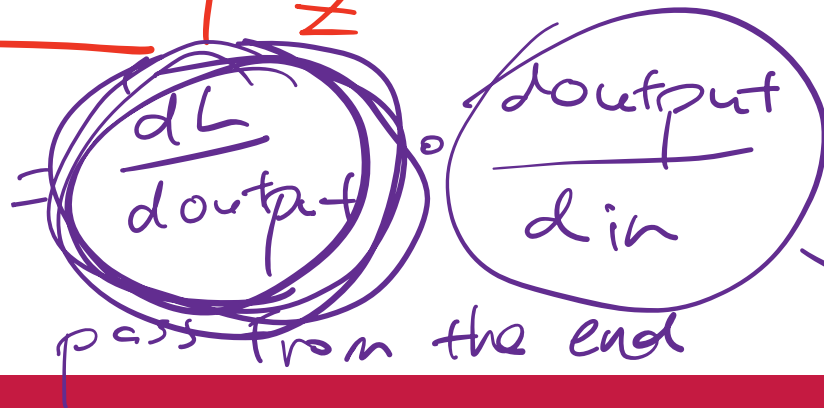
$$\frac{df}{dx} = \frac{df}{dy} \cdot \frac{dy}{dx}$$

in

out



$$\frac{dL}{dw}$$



Neural Network Learning Process

Neural networks learn using gradient descent as an iterative process that finds the minimum point by taking steps in the direction of steepest descent.

High-level steps

1. **Forward pass:** run batch through network with current weights/biases
2. **Calculate loss** (Sum of Squared Residuals / Sum of Predicted - Actual)
3. **Backward pass:** calculate gradients of loss with respect to each weight/bias parameter

Prediction = intercept + slope × weight

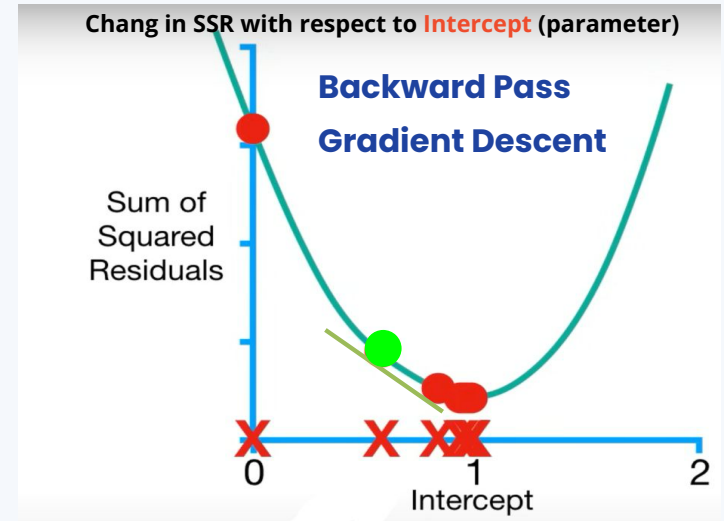
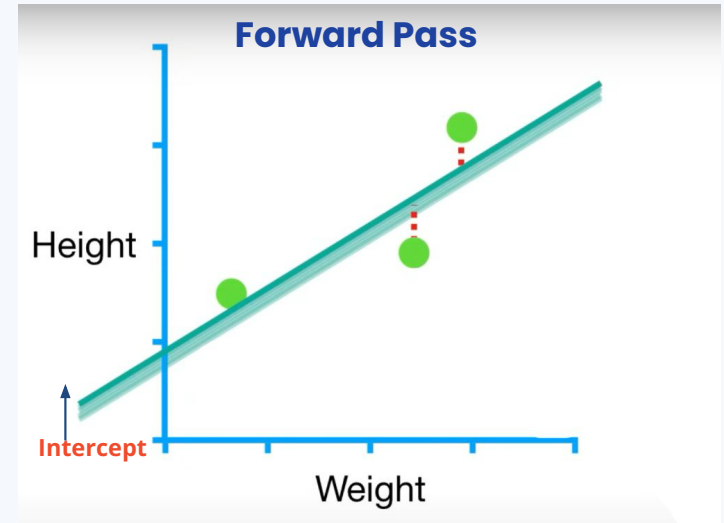
Loss = (prediction - actual)²

$$\frac{\partial \text{Loss}}{\partial \text{Prediction}} = 2 \times (\text{prediction} - \text{actual})$$

$$\frac{\partial \text{Prediction}}{\partial \text{Intercept}} = 1$$

$$\frac{\partial \text{Loss}}{\partial \text{Intercept}} = \frac{\partial \text{Loss}}{\partial \text{Prediction}} \times \frac{\partial \text{Prediction}}{\partial \text{Intercept}} = 2 \times (\text{prediction} - \text{actual}) \times 1$$

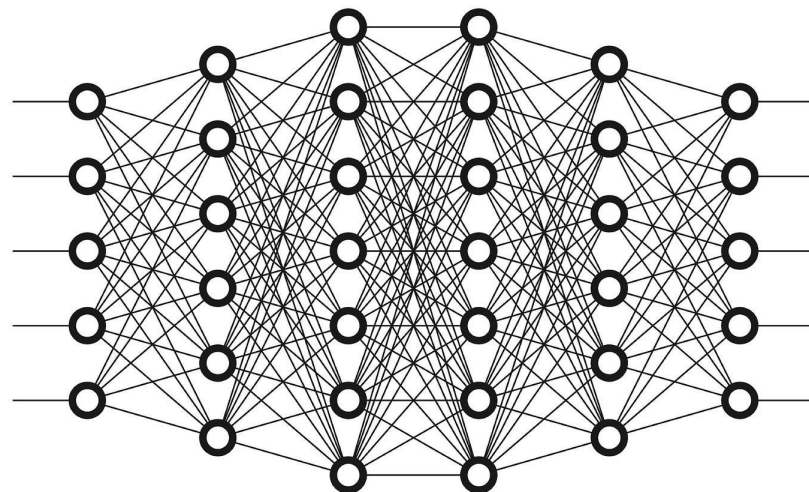
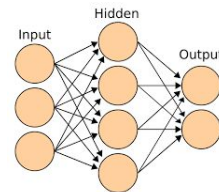
4. **Gradient descent:** Update each parameter
new_value = old_value - (learning_rate × gradient)
5. **Repeat** with next batch until change in loss is small



Building a Neural Network (1/4)

General Guidelines for Number of Layers

- **Shallow** networks (1-2 hidden layers) are often enough for simpler datasets like MNIST, where the task is relatively straightforward.
- **Deeper** networks (3-5 layers) are typically used for more complex datasets where hierarchical feature extraction is essential (e.g., in image classification or speech recognition).
- **Very deep** networks (6+ layers) can be useful for highly complex problems, but this often requires more data, more computation, and better regularization techniques (e.g., dropout, batch normalization).

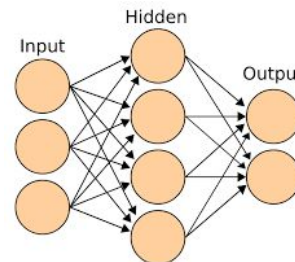


L1, L2 norm

Building a Neural Network (2/4)

General Guidelines for Number of Neurons per Layer

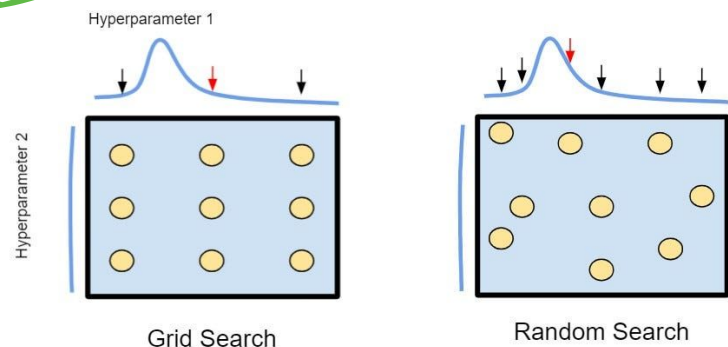
- **Start small:** A good initial guess is to use 64, 128, or 256 neurons in the first hidden layer.
- **Reduce neurons in subsequent layers:** As the network learns more abstract representations, it's common to reduce the number of neurons in subsequent layers (e.g., $256 \rightarrow 128 \rightarrow 64$).
- **Scaling:** For image classification, you might start with 512 or 1024 neurons in the first layer and then gradually reduce the number of neurons.
- **Keep the number of neurons balanced:** If you have too few neurons, the model might be too simple to capture complex patterns. Too many neurons can cause overfitting and longer training times.



Building a Neural Network (3/4)

Hyperparameter Search and Tuning

- **Grid Search:** You can perform a grid search over different architectures (number of layers and neurons) to find the best combination.
- **Random Search:** An alternative to grid search, random search involves randomly selecting hyperparameters within a predefined range.
- **Bayesian Optimization:** This is a more advanced method for hyperparameter tuning that tries to intelligently search for the best architecture by focusing on promising hyperparameters.



<https://towardsdatascience.com/hyperparameter-tuning-always-tune-your-models-7db7aeaf47e9/>

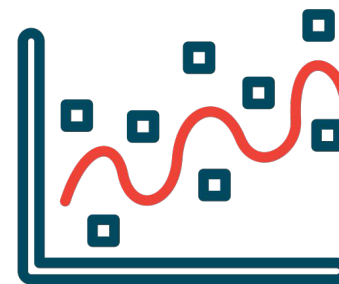
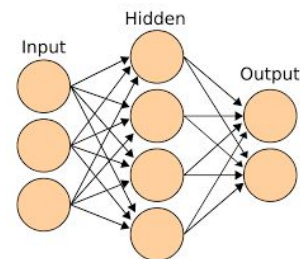
Building a Neural Network (4/4)

Monitor Model Performance and Adjust

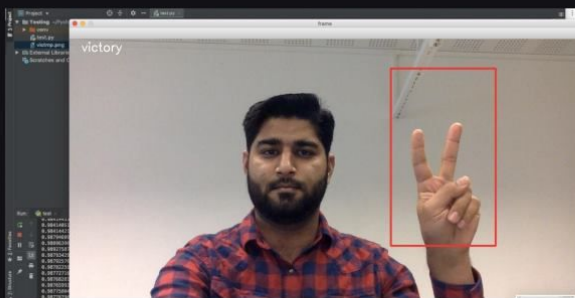
- **Underfitting:** If your model is not learning well or the training error is high, increase the number of neurons or add more layers to capture more complex patterns.
- **Overfitting:** If your model is performing well on training data but poorly on validation/test data, you might have too many neurons or layers, causing the model to memorize the data.

You can try:

- Reducing the number of neurons or layers.
- Using regularization techniques like dropout.
- Using early stopping to avoid overfitting.



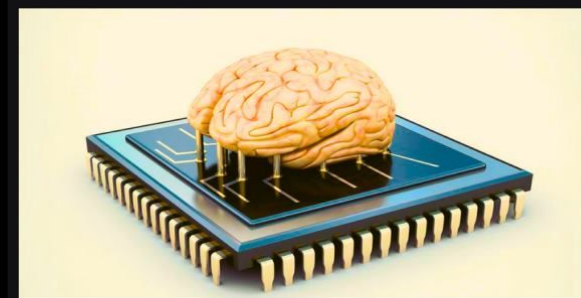
DEEP LEARNING



WHAT MY FRIENDS THINK I DO



WHAT MY MOM THINKS I DO



WHAT SOCIETY THINKS I DO

Cost function

Logistic regression:

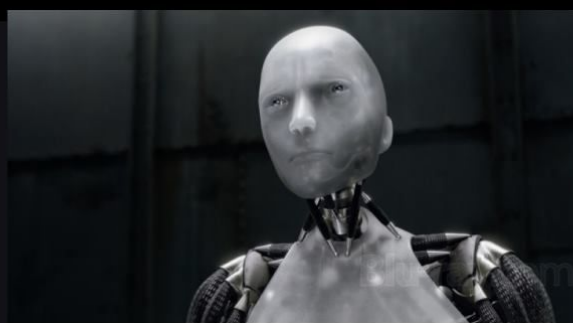
$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Neural network:

$\rightarrow h_{\Theta}(x) \in \mathbb{R}^K$ ($h_{\Theta}(x)$)_i = ith output

$$\rightarrow J(\Theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log (h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log (1 - (h_{\Theta}(x^{(i)}))_k) \right] + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2$$

WHAT INTERNS THINKS I DO

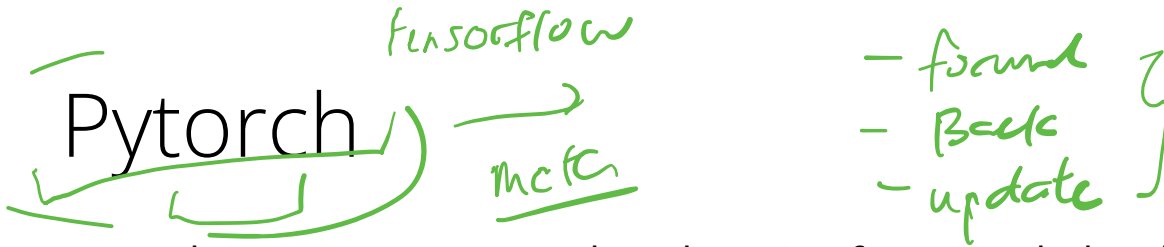


WHAT I THINK I DO

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

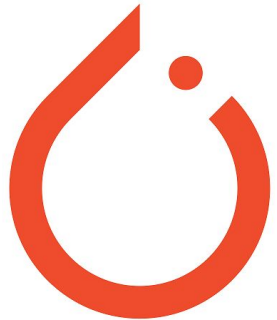
WHAT I REALLY DO

By: Spotter - Search & Compare Courses



PyTorch is an open-source deep learning framework developed by Meta's AI Research lab. It provides a flexible and efficient platform for building and training deep learning models. PyTorch has gained significant popularity because of its ease of use, dynamic computation graph, and integration with Python, making it a go-to framework for research and production in the AI community.

It's commonly used in applications like image recognition and language processing. Written in Python, it's relatively easy for most machine learning developers to learn and use. PyTorch is distinctive for its excellent support for GPUs and its use of reverse-mode auto-differentiation, which enables computation graphs to be modified on the fly. This makes it a popular choice for fast experimentation and prototyping.



Who's using Pytorch?

Amazon uses PyTorch to flag ads that don't comply with the content guidelines of its advertising service. They use PyTorch to develop computer vision and natural language processing models that automatically flag potentially noncompliant ads.

Integrating PyTorch with their internal infrastructure allows Amazon to provide an optimal customer experience and create ML models that are fast enough to serve ads in milliseconds.

Meta uses PyTorch to train its language translation system that handles **6 billion translations** a day and to power its Instagram recommendation system that suggests new content (Feeds, Stories, or Reels) that match your taste.

Tesla uses PyTorch to train and deploy deep learning models for Autopilot—their **self-driving** technology—and to power features like lane-keeping assistance, object detection, and Smart Summon for their cars.



<https://www.altexsoft.com/blog/pytorch-library/>

Auto Grad



Types of Deep Learning Architectures

Model	Use Cases	Key Characteristic
Feedforward Neural Networks (FNNs)	Classification, regression tasks, simple tabular data	Data flows in one direction (forward) from input to output.
Convolutional Neural Networks (CNNs)	Image classification, object detection, image segmentation, video analysis	Designed for grid-like data (e.g., images), captures spatial hierarchies with convolution filters.
Recurrent Neural Networks (RNNs)	Time series prediction, NLP, speech recognition, sequence-to-sequence tasks (e.g., translation)	Good for sequential data, where output depends on previous time steps.
Generative Adversarial Networks (GANs)	Image generation, style transfer, data augmentation, deepfakes	Generates synthetic data through competition between two networks (generator vs discriminator).
Transformer Networks	NLP tasks (e.g., machine translation, summarization), text generation	Handles long-range dependencies efficiently with self-attention, allows for parallelization.

PyTorch, Keras, Tensorflow Framework Comparison

Feature	Keras	TensorFlow	PyTorch
API Level	High-level	Low + High-level	Low-level
Syntax	Simple	Moderate	Pythonic, Flexible
Computation Graph	Static (via TF)	Static + Eager	Dynamic
Popular Use	Prototyping	Production	Research
Debugging	Limited	TensorBoard	Native Python
Ecosystem	Integrated	Extensive	Growing



Keras

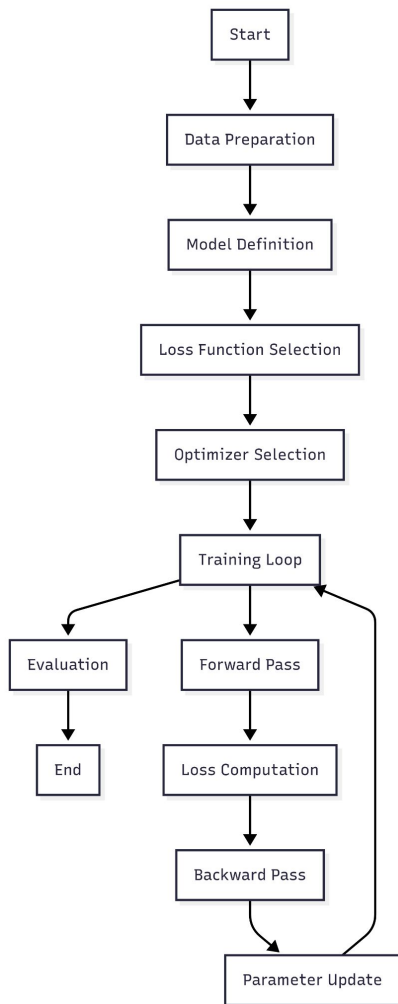


TensorFlow



PyTorch

Generalized Code



```
import tensorflow as tf

# 1. Prepare your data
# Example: Load a sample dataset (e.g., MNIST)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0 # Normalize pixel values

# 2. Build the model
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)), # Input layer
    tf.keras.layers.Dense(128, activation='relu'), # Hidden layer
    tf.keras.layers.Dense(10, activation='softmax') # Output layer
])

# 3. Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 4. Train the model
model.fit(x_train, y_train, epochs=5)

# 5. Evaluate the model
model.evaluate(x_test, y_test)
```



TensorFlow

```
import torch
import torch.nn as nn
import torch.optim as optim

# 1. Define the model by subclassing nn.Module
class SimpleModel(nn.Module):
    def __init__(self):
        super(SimpleModel, self).__init__()
        self.layer1 = nn.Linear(28*28, 128) # Example: input size for MNIST
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(128, 10) # Output size for 10 classes

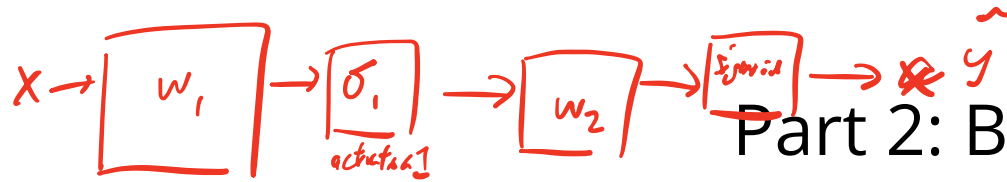
    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x

# 2. Instantiate the model, loss function, and optimizer
model = SimpleModel()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 3. Example training loop
for epoch in range(5): # Number of epochs
    model.train()
    for batch_X, batch_y in dataloader: # Replace 'dataloader' with your DataLoader
        optimizer.zero_grad()
        outputs = model(batch_X)
        loss = loss_fn(outputs, batch_y)
        loss.backward()
        optimizer.step()
```



PyTorch



Part 2: Building a MLP (Fully Connected Neural Network) with PyTorch

$$\text{sigmoid}(w_2(\sigma_1(w_1^T \cdot x)))$$

End of Lesson - Exit Ticket

Survey at

Slido link will be provided in class

